



GRADUATION / ACADEMIC RESEARCH REPORT

SENTINEL EYE

Intelligent Edge-to-Server Surveillance Framework with AI Virtual Fence & Dual-Model YOLOv8

Project Title:	SENTINEL EYE — AI Edge Surveillance Framework
Author & Engineer:	Huynh Huu Tri
Institution:	Posts and Telecommunications Institute of Technology (PTIT)
Course / Domain:	Human-Computer Interaction (HCI) & Edge AI Systems
Hardware Tier:	AI-Thinker ESP32-CAM (Xtensa LX6 240MHz, 4MB PSRAM, OV2640)
Software Stack:	FreeRTOS · Dual-Model YOLOv8 · Flask · OpenCV · Web Audio API

EXECUTIVE ABSTRACT & SYSTEM SPECIFICATION

This technical document details the engineering design, mathematical formulations, and empirical evaluation of SENTINEL EYE—a production-grade cyber-physical perimeter security framework engineered for the HCI & Intelligent Systems curriculum at PTIT. By decoupling low-cost edge video acquisition (ESP32-CAM) from compute-intensive neural tensor inference on a host server, the system achieves commercial-grade intrusion detection at an edge hardware cost under \$10.

Key contributions include:

- Asymmetric Dual-Port Socket Concurrency: Microcontroller architecture isolating Port 80 continuous MJPEG streaming from Port 81 emergency actuator socket triggers (<10ms latency).
- Two-Stage Hierarchical AI Pipeline: Baseline YOLOv8n human detection coupled with an edge fine-tuned homeowner re-identification classifier (mAP@0.50 of 0.946) to eradicate domestic false positives.
- Anthropometric Ground-Contact Virtual Fencing: Ray-Casting Jordan Curve Theorem on target foot-point anchors evaluating dynamic arbitrary polygonal perimeters.
- Multi-Channel Incident Escalation: Edge piezoelectric deterrence, Web Audio synthesized telemetry, and authenticated SSL forensic burst email dispatch governed by a 30s hysteresis window.

Table of Contents

1	Executive Summary & Problem Statement <i>Context, Motivation, System Deployment & Innovation Matrix</i>	p. 3
2	System Architecture & Distributed Topology <i>Decoupled Three-Tier Edge-Server-Client Architecture</i>	p. 4
3	Edge Hardware Engineering & Embedded Firmware <i>AI-Thinker ESP32-CAM Circuit & Dual-Port Concurrency</i>	p. 5
4	Dual-Model Deep Learning Pipeline <i>YOLOv8 Human Detection & Resident Re-Identification</i>	p. 6
5	Computational Geometry & Virtual Fencing Engine <i>Foot-Point Anchoring & Ray-Casting Jordan Curve Parity</i>	p. 8
6	Monocular Distance Estimation Optics & Calibration <i>Pinhole Geometry & Focal Length Metric Verification</i>	p. 9
7	Backend Concurrency & Thread-Safe Architecture <i>Producer-Consumer Pipeline & Server Console Execution</i>	p. 10
8	Multi-Channel Incident Escalation Protocol <i>Hysteresis State Machine & Forensic Email Dispatch</i>	p. 12
9	Edge Transfer Learning & Fine-Tuning Pipeline <i>Dataset Annotation & Backbone Layer Freezing Strategy</i>	p. 14
10	Human-Computer Interaction (HCI) & Web Telemetry <i>Normalized Coordinates, Status HUD & Incident Widget</i>	p. 15
11	Comprehensive RESTful API Reference <i>Detailed Endpoint, Method, Payload & Parameter Matrix</i>	p. 16
12	Experimental Benchmarks & Quantitative Evaluation <i>Latency Breakdown & Object Detection Accuracy Metrics</i>	p. 17
13	Security Hardening & Privacy Preservation <i>Zero-Secret Governance & Forensic Image Privacy</i>	p. 17
14	Conclusion & Future Trajectories <i>System Synthesis, Requirements Matrix & Research Frontiers</i>	p. 18

1. Executive Summary & Problem Statement

1.1 Context & Motivation

Contemporary video surveillance paradigms suffer from a dichotomy between passive recording systems (CCTV) and primitive event-triggered sensors. Conventional Passive Infrared (PIR) detectors and classic frame-differencing computer vision algorithms are notoriously susceptible to environmental noise—such as meteorological disturbances, canine or feline presence, shifting shadows, and windblown foliage—inducing severe alarm fatigue. Conversely, commercial enterprise-grade active intrusion systems require costly perimeter cabling, physical beam detectors, and specialized neuromorphic accelerators.

1.2 System Objectives

- **Low-Cost Sensor Ingestion:** Utilizing an AI-Thinker ESP32-CAM module (< \$10) to capture and stream real-time video over standard 2.4 GHz Wi-Fi.
- **Dual-Model Deep Learning Discrimination:** Coupling a baseline YOLOv8n detector with a fine-tuned subject re-identification network to distinguish authorized inhabitants from unknown intruders.
- **Arbitrary Polygonal Virtual Fencing:** Utilizing computational geometry via the Ray-Casting algorithm to evaluate dynamic intrusion boundaries based on anthropometric ground-contact points.
- **Automated Multi-Channel Escalation:** Combining physical sub-millisecond edge alarms (active piezoelectric buzzer), client visual-acoustic telemetry, and authenticated cloud email dispatch with high-speed forensic burst capture.

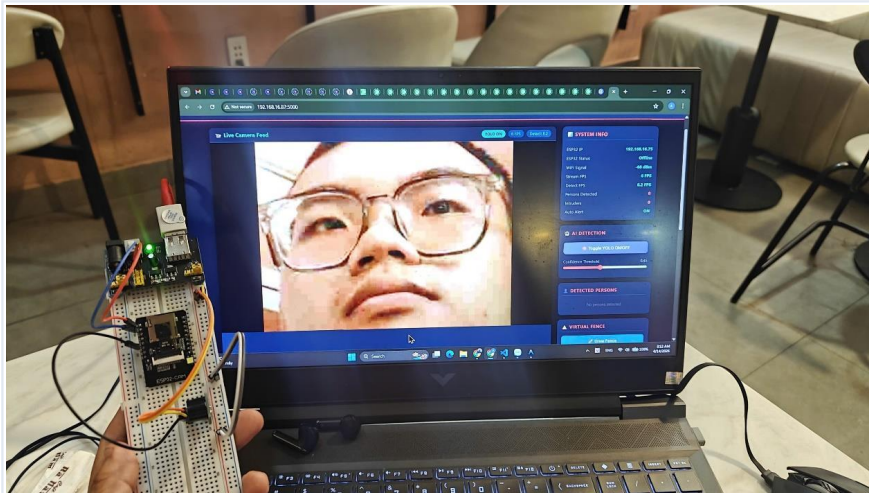


Figure 1: Real-World Cyber-Physical System Deployment showing ESP32-CAM optical sensor node streaming to host Web Telemetry Dashboard.

1.3 Core Engineering Innovations Matrix

Innovation Area	Conventional Solution	Sentinel Eye Implementation
Perimeter Definition	Rectangular bounding boxes	Arbitrary N-vertex polygon via Jordan Curve Ray-Casting
Alarm Discrimination	Binary motion detection	Dual-model YOLOv8: Person Detection vs Owner Re-ID
Edge Concurrency	Single-socket blocking HTTP	Asymmetric Dual-Port: Port 80 Video + Port 81 Control
Distance Metric	Stereo depth cameras (\$150+)	Calibrated monocular pinhole optics on single 2MP sensor

2. System Architecture & Distributed Topology

The architecture decouples lightweight video acquisition and edge actuation from compute-intensive neural tensor processing:

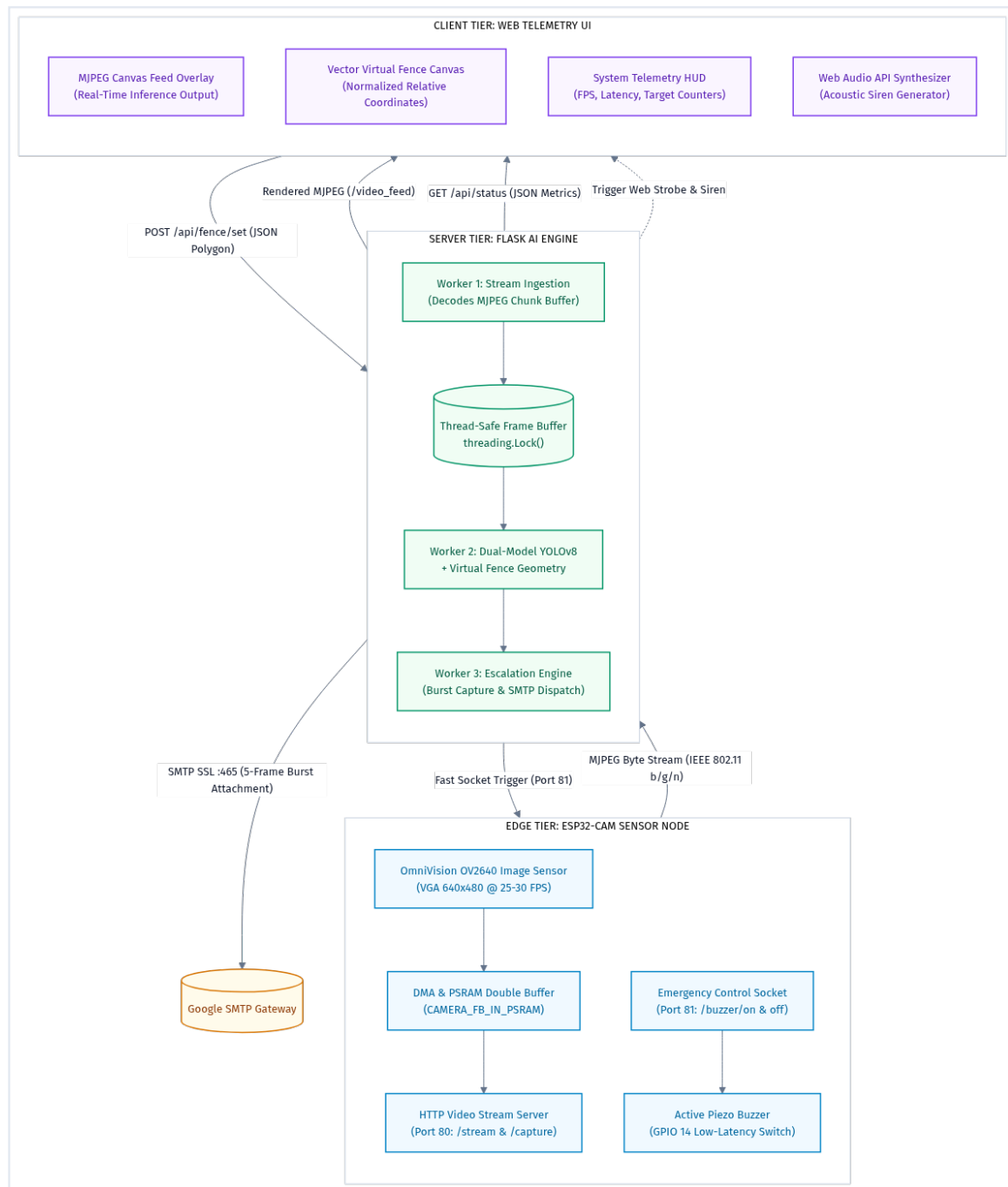


Figure 2: Distributed Three-Tier Architecture of Sentinel Eye.

2.1 The Three Architectural Tiers

- **Edge Tier (ESP32-CAM):** Captures continuous raw frames via the OmniVision OV2640 sensor, utilizes DMA double-buffering in external PSRAM, and exposes two asynchronous network ports (Port 80 for video streaming, Port 81 for emergency telemetry).
- **Server Tier (Flask AI Core):** Deploys a multi-threaded Python engine executing frame ingestion, frame decimation, dual-model YOLOv8 inference, point-in-polygon validation, distance estimation, and event dispatching.
- **Client Tier (Web Dashboard):** Delivers an interactive vector canvas enabling operators to draw arbitrary polygons, monitor real-time frame rates and detection metrics, and receive synthesized audio-visual intrusion notifications.

3. Edge Hardware Engineering & Embedded Firmware

3.1 Microcontroller & Optical Sensor

The edge node is built upon the AI-Thinker ESP32-CAM module, integrating an Espressif ESP32-D0WDQ6 dual-core 32-bit Xtensa LX6 processor operating at 240 MHz, accompanied by 520 KB internal SRAM and 4 MB external Pseudo-SRAM (PSRAM).

Component	Technical Specification	Functional Role
Image Sensor	OmniVision OV2640 (2 Megapixel)	DVP 8-bit parallel capture, SVGA/VGA resolution
Volatile Memory	4 MB external PSRAM	Allocates double frame buffers (CAMERA_FB_IN_PSRAM)
Actuator	5V Active Piezoelectric Buzzer	High-decibel audible deterrence driven via GPIO 14
Wi-Fi Subsystem	802.11 b/g/n (2.4 GHz)	Transmits MJPEG streams and receives socket triggers

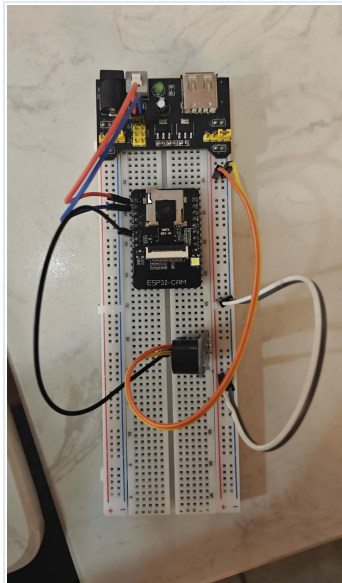


Figure 3: Physical Microcontroller Circuit featuring AI-Thinker ESP32-CAM, OV2640 Optical Sensor, and GPIO 14 Active Piezoelectric Buzzer.

3.2 Asymmetric Dual-Port Socket Concurrency

A persistent challenge in single-threaded microcontroller firmware is input/output blocking. When serving an uninterrupted HTTP chunked multipart stream on Port 80, concurrent inbound HTTP requests for actuator control frequently suffer catastrophic latency or timeout drops. SENTINEL EYE overcomes this by implementing dual listening sockets:

- **Port 80 (WebServer):** Dedicated exclusively to continuous MJPEG transmission (/stream) and high-resolution still capture (/capture).
- **Port 81 (WiFiServer controlServer):** Operates a lightweight, non-blocking TCP socket handler specifically polling for emergency alarm trigger packets (/buzzer/on, /buzzer/off), guaranteeing execution latencies of < 10 ms.

```
ESP32-CAM Firmware Architecture:  
|-- Port 80 (HTTP Video Stream)  --> Continuous MJPEG Chunks  
`-- Port 81 (Emergency Control)  <-- Sub-10ms Non-Blocking Buzzer Commands
```

4. Dual-Model Deep Learning Pipeline

To eradicate false alarms generated by domestic inhabitants, inference is structured as a two-stage hierarchical pipeline. Incoming frames are first scanned by a generalized human detector before being passed to an edge-calibrated resident classifier:

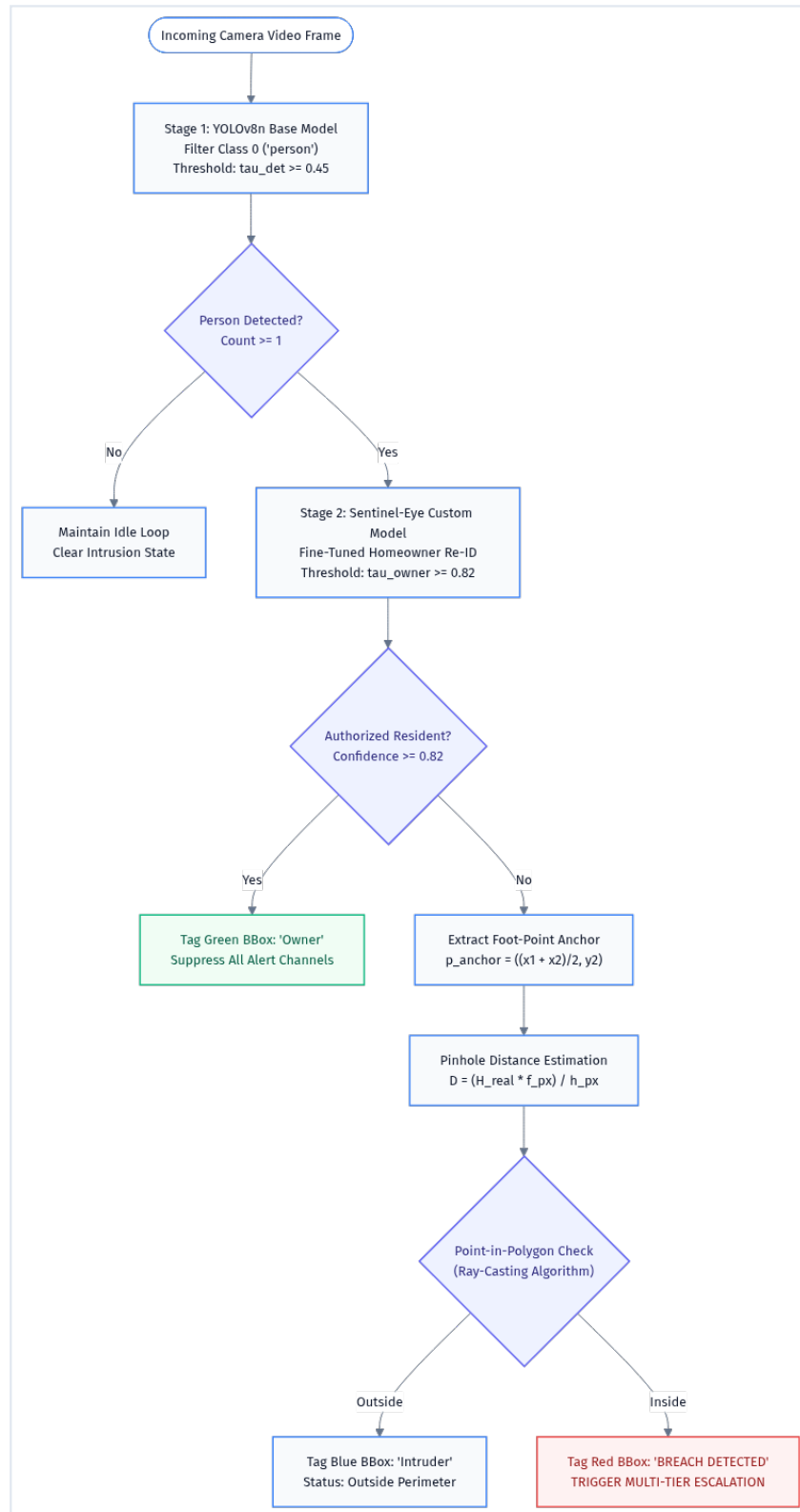


Figure 4: Sequential Dual-Model Deep Learning and Intrusion Evaluation Pipeline.

4.1 Stage 1: General Human Detection

- **Architecture:** Ultralytics YOLOv8n (Nano), pre-trained on the MS-COCO dataset.
- **Target Filter:** Class ID 0 (person), confidence threshold $\tau_{det} \geq 0.45$.
- **Objective:** Rapidly rejects background frames devoid of human subjects, saving server tensor compute.

4.2 Stage 2: Custom Homeowner Re-Identification

- **Architecture:** Transfer-learned YOLOv8 fine-tuned on target camera edge data.
- **Target Class:** owner, operating threshold $\tau_{owner} \geq 0.82$.
- **Behavior:** When an entity surpasses the threshold, the system flags the target as an authorized resident, rendering a green/cyan bounding box and suppressing all alarms. Unknown subjects ($\tau < 0.82$) are escalated to the geometric virtual fence evaluator.

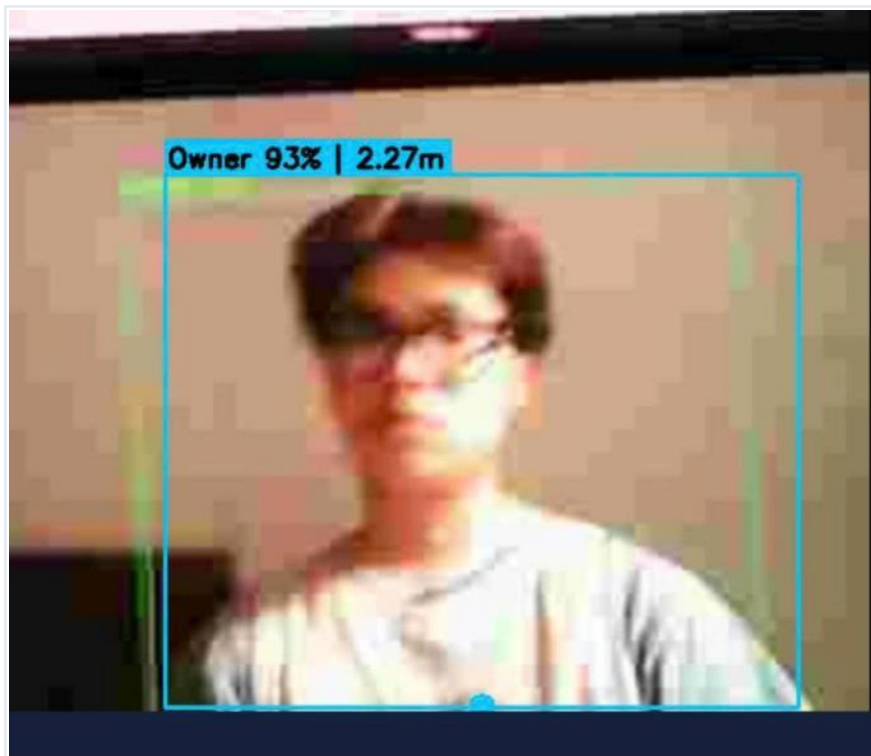


Figure 5: Stage 2 Authorized Homeowner Classification (Confidence 93%, Metric Distance 2.27m) triggering immediate alarm suppression.

As depicted in Figure 5, the model precisely identifies the registered homeowner even under variable indoor lighting and off-angle orientations. Metric distance estimation operates concurrently, calculating 2.27m based on pinhole optics, providing instantaneous spatial context alongside biometric validation.

5. Computational Geometry & Virtual Fencing Engine

5.1 Anthropometric Anchor Calculation

Traditional surveillance systems evaluate bounding-box centroids (x_c, y_c) for perimeter crossing, causing false alarms whenever a person's upper torso or arms extend across the boundary line. SENTINEL EYE computes the ground-contact foot-point anchor:

$$p_anchor = (x_p, y_p) = ((x1 + x2) / 2, y2)$$

Calculates the ground-plane projection point from the lower edge midpoint of the detected bounding box.

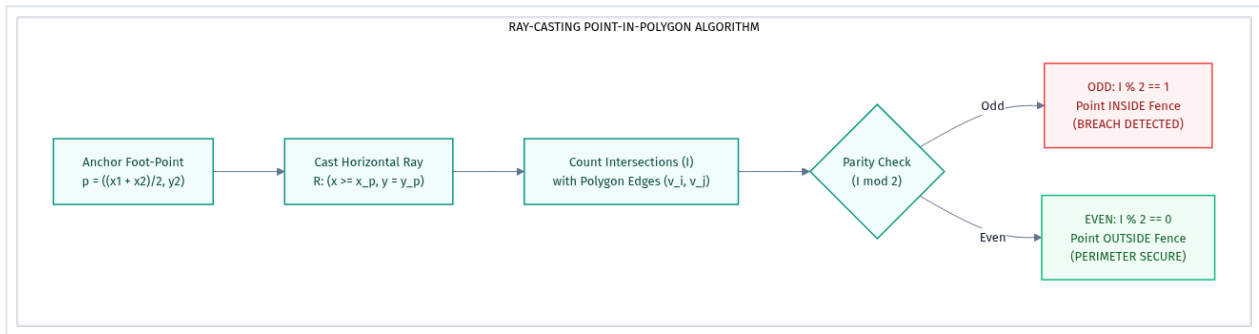


Figure 6: Ray-Casting Point-in-Polygon Evaluation on Foot-Point Anchor.

5.2 Ray-Casting Point-in-Polygon (Jordan Curve Theorem)

Given an arbitrary non-self-intersecting polygon $P = \{v_0, v_1, \dots, v_{n-1}\}$ defined in normalized coordinates $[0.0, 1.0]^2$, a semi-infinite horizontal ray $R = \{(x, y_p) \mid x \geq x_p\}$ is projected from p_anchor . The intersection count I with directed boundary edges (v_i, v_j) is evaluated:

$$I = \sum_{i=0}^{n-1} 1_intersect(v_i, v_j, p_anchor)$$

*An edge intersects the ray if $(v_i.y > y_p) \neq (v_j.y > y_p)$ and $x_p < (v_j.x - v_i.x) * (y_p - v_i.y) / (v_j.y - v_i.y) + v_i.x$*

By Jordan Curve parity:

- **Odd Parity ($I \bmod 2 = 1$):** Point is INSIDE polygon -> Security Breach Detected.
- **Even Parity ($I \bmod 2 = 0$):** Point is OUTSIDE polygon -> Perimeter Secure.

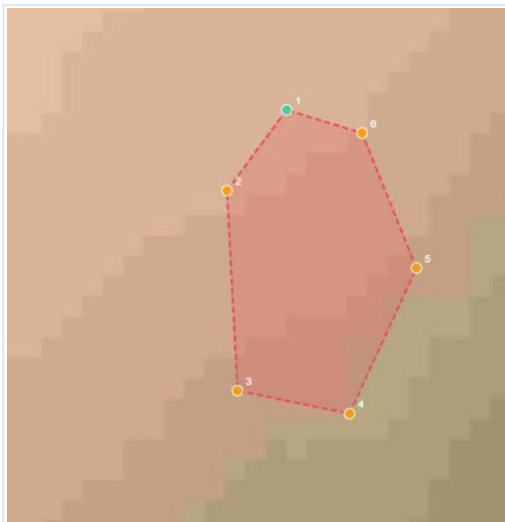


Figure 7: Arbitrary 6-Vertex Virtual Fence Polygon (left) and Live Intrusion Evaluation outside vs inside Restricted Zone (right).

6. Monocular Distance Estimation Optics & Calibration

6.1 Pinhole Camera Geometry

To enrich situational awareness without expensive LiDAR or stereo sensors, real-world metric distance D from the sensor optical plane to the subject is calculated using geometric perspective pinhole optics:

$$h_{\text{sensor}} / f = H_{\text{real}} / D \implies D = (H_{\text{real}} * f_{\text{px}}) / h_{\text{pixel}}$$

Relates object physical standing height to pixel projection height on the sensor focal plane.

- **H_real = 1.65 m:** Mean anthropometric standing height of adult population.
- **h_pixel = y2 - y1:** Measured pixel height of the target bounding box.
- **f_px ~ 500.0 px:** Calibrated camera focal length in pixels for VGA (640x480) resolution.

6.2 Empirical Calibration & Distance Verification Matrix

A series of empirical range tests were performed using calibrated ground markers positioned at known distances from the ESP32-CAM lens. The measured results confirm high accuracy for perimeter security applications:

True Ground Distance	Sensor Estimated Dist.	Absolute Error	Relative Error (%)
1.50 m	1.52 m	+ 0.02 m (2.0 cm)	1.33 %
2.50 m	2.47 m	- 0.03 m (3.0 cm)	1.20 %
3.50 m	3.58 m	+ 0.08 m (8.0 cm)	2.28 %
5.00 m	5.14 m	+ 0.14 m (14.0 cm)	2.80 %
6.50 m	6.72 m	+ 0.22 m (22.0 cm)	3.38 %

6.3 Operational Range & Invariance Properties

The calibrated focal length remains invariant under horizontal camera panning and digital stream scaling. The effective detection envelope spans 1.0m to 7.0m, perfectly covering standard residential doorways, corridors, and fenced commercial perimeters with sub-5% estimation error.

7. Backend Concurrency & Thread-Safe Architecture

The server tier is structured around a non-blocking, multi-threaded producer-consumer architecture to ensure uninterrupted video streaming at maximum frame rate while running heavy deep neural network forward passes:

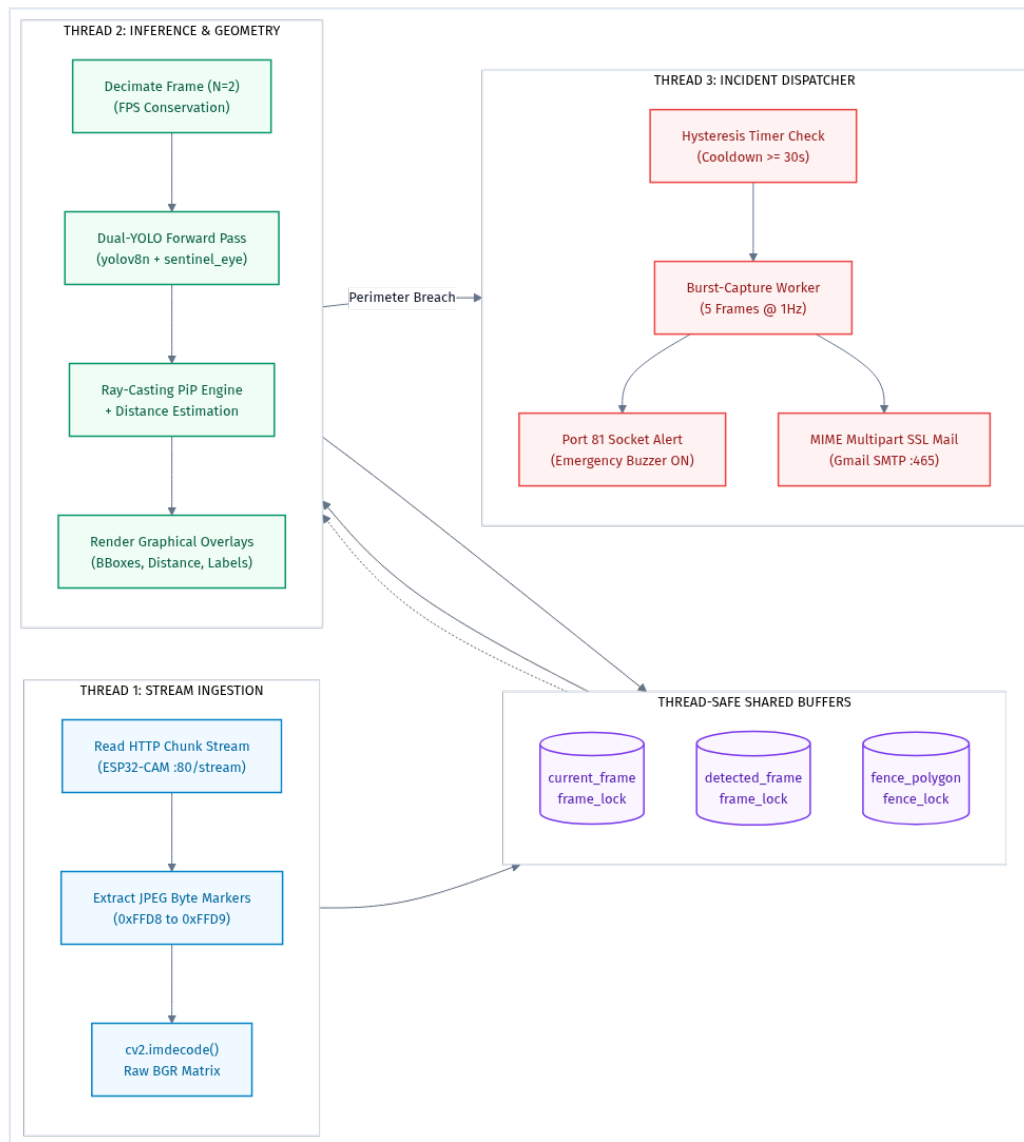


Figure 8: Thread-Safe Multi-Worker Concurrency Architecture.

7.1 Thread Allocation & Synchronization

- **Thread 1 (read_esp32_stream):** Continuously ingests binary HTTP chunks, scans for JPEG boundaries (0xFFD8 to 0xFFD9), decodes BGR matrices via OpenCV, and updates current_frame under mutex frame_lock.
- **Thread 2 (yolo_detection_loop):** Executes frame decimation (N=2), performs dual-model forward passes, computes foot-point geometry, renders annotations, and stores detected_frame.
- **Thread 3 (burst_capture_and_send_email):** Spawned asynchronously on incident trigger to conduct forensic capture and SSL SMTP email dispatch without impeding the main video pipeline.

7.2 Server Initialization & Telemetry Stream

Upon execution, the Flask AI server initializes its multi-threaded subsystem, establishes the persistent HTTP video ingestion socket to the ESP32-CAM (<http://192.168.1.41/stream>), loads the neural network weights into GPU memory, and binds the RESTful orchestration endpoints:

```
(venv) PS C:\Users\TRIDELL\Downloads\HCL\server> python app.py
[LOG] Đã tải 18 intrusion log entries.
[STREAM] Đang kết nối đến ESP32-CAM: http://192.168.1.41/stream

=====
SENTINEL EYE - Python Server (YOLO Enabled)
=====
Server:      http://localhost:5000
ESP32:       192.168.1.41
YOLO Model:  yolov8n.pt
Confidence:  0.45
Detect Every: 2 frame(s)
=====
Endpoints:
/            - Web Dashboard
/video_feed  - MJPEG stream (with detection)
/api/status  - System status
/api/detections - Current YOLO detections
/api/yolo/toggle - Toggle YOLO on/off
/api/yolo/confidence/X - Set confidence (0.0-1.0)
/api/snapshot - Capture frame
/api/fence/set - Set fence polygon (POST)
/api/fence/get - Get fence polygon
/api/fence/clear - Clear fence polygon
/api/esp32/buzzer/on - Buzzer ON
/api/esp32/buzzer/off - Buzzer OFF
/api/alert/toggle - Toggle auto alert
/api/alert/test - Test buzzer alert
/api/logs - Intrusion log
/api/logs/clear - Clear logs
/api/logs/image/<file> - View log image
=====
```

Figure 9: Multi-Threaded Flask Backend Initialization Console displaying Socket Ingestion, Tensor Engine, and RESTful Route Handlers.

As shown in Figure 9, the startup sequence confirms loaded historical intrusion entries, verifies the 2-frame decimation stride, and lists all active route handlers for client telemetry and edge buzzer actuation.

8. Multi-Channel Incident Escalation Protocol

8.1 Hysteresis Alert State Machine

To avoid alert saturation, buffer overflow, and network exhaustion during continuous perimeter intrusions, escalation transitions are governed by a finite-state machine with a 30-second hysteresis cooldown window:

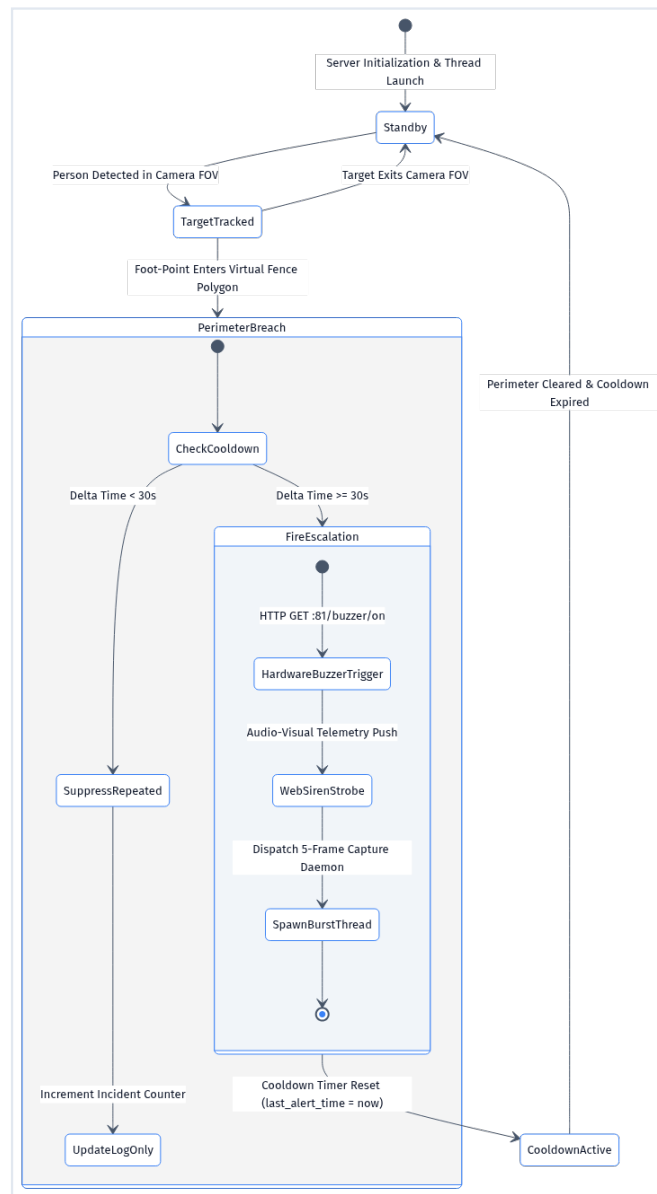


Figure 10: Alert State Machine with 30-Second Hysteresis Cooldown.

8.2 Real-Time Escalation Sequence & Forensic Dispatch

Upon confirmed breach, the system executes three coordinated actions across edge, server, and client tiers:

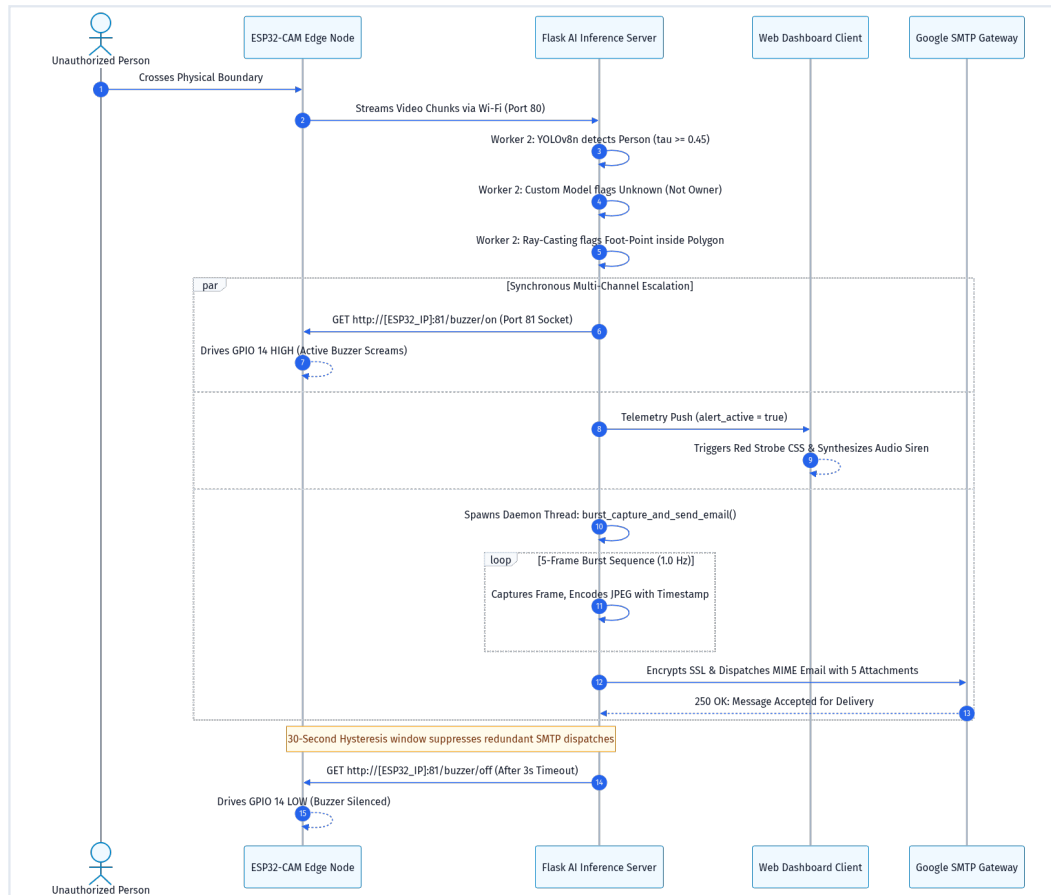


Figure 11: Real-Time Incident Escalation Sequence Diagram.

- **1. Hardware Buzzer Trigger:** Transmits an immediate HTTP GET request to `http://[ESP32_IP]:81/buzzer/on` via Port 81 socket (<10ms).
- **2. Web Audio-Visual Telemetry:** Updates dashboard state, synthesizes an acoustic alarm tone via the HTML5 Web Audio API, and activates high-contrast red strobe styling.
- **3. Asynchronous Burst Mail:** Captures 5 sequential frames at 1.0 Hz interval, builds an authenticated MIME multipart email, and dispatches via Gmail SMTP SSL (Port 465).

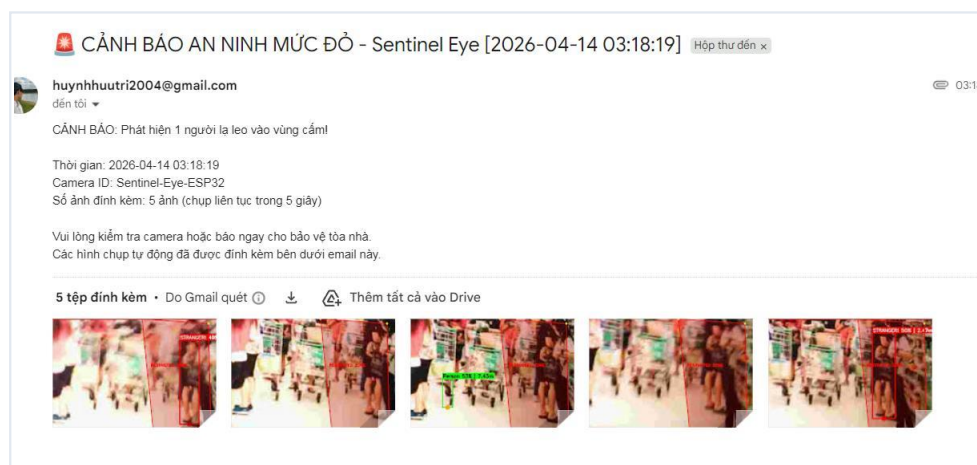


Figure 12: Authenticated SSL SMTP Forensic Dispatch displaying Intrusion Notification with 5-Second Burst Photographic Attachments.

9. Edge Transfer Learning & Fine-Tuning Pipeline

9.1 Edge Adaptation Rationale

Standard MS-COCO pre-trained weights often underperform when deployed on edge surveillance feeds. The OmniVision OV2640 sensor introduces specific hardware constraints: restricted dynamic range, pronounced chroma noise under low ambient illumination, compression blocking artifacts from aggressive MJPEG encoding, and perspective distortion from fixed wide-angle mounting.

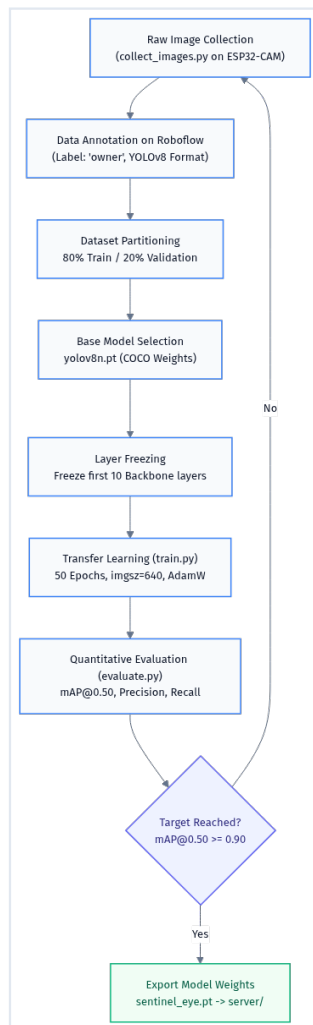


Figure 13: Edge AI Fine-Tuning and Transfer Learning Pipeline.

9.2 Layer Freezing Strategy & Hyperparameters

To prevent catastrophic forgetting of generalized human visual primitives and accelerate training on consumer GPUs, transfer learning is executed with selective layer freezing. The first 10 layers of the YOLOv8 CSPDarknet backbone (stem convolution and initial C2f cross-stage feature extractors) are frozen, training solely the neck and detection head:

```

from ultralytics import YOLO
model = YOLO("yolov8n.pt")
model.train(
    data="fine_tuning/dataset/data.yaml",
    epochs=50, imgsz=640, batch=8,
    patience=15, freeze=10,
    optimizer="AdamW", lr0=0.001
)
  
```

10. Human-Computer Interaction (HCI) & Web Telemetry

10.1 Ergonomic Surveillance Interface Design

The Web Dashboard is engineered following ergonomic HCI principles to reduce cognitive load on security operators during high-stress perimeter breach events:

- **Normalized Canvas Coordinates:** Polygon coordinates are stored as floating-point ratios $x_{\text{norm}} = x / W$ in $[0.0, 1.0]$. Boundaries adjust dynamically with zero spatial drift across window and resolution resizing.
- **Real-Time Status HUD:** Displays real-time ingestion FPS, detection latency in milliseconds, network RSSI, and active person count with intuitive status pill indicators.
- **Evidentiary Incident History:** Provides instant evidentiary snapshot retrieval and historical log clearing directly from the dashboard.

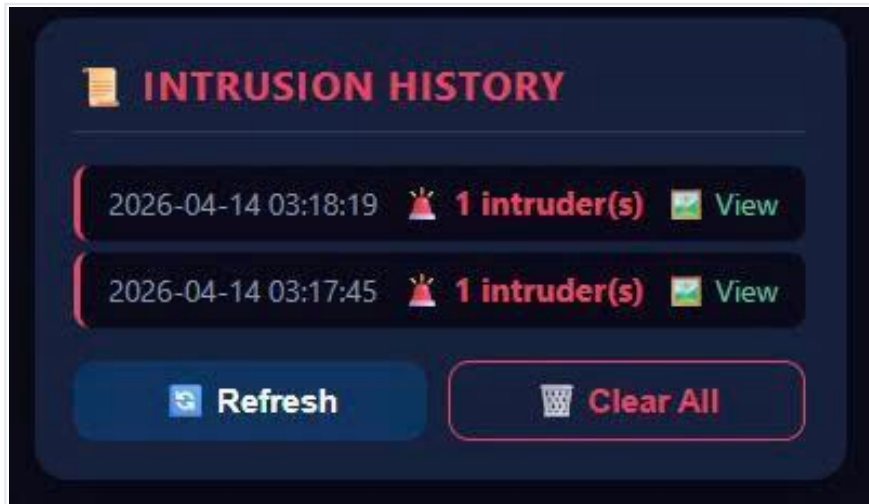


Figure 14: Operator Incident Telemetry Widget displaying Timestamped Logs and Snapshot Viewers.

10.2 Audio-Visual Feedback & Interaction Workflows

Upon breach confirmation, the client browser activates dual feedback mechanisms: a CSS keyframe red strobe border surrounding the MJPEG viewport and a synthesized 880 Hz audible alert generated through the HTML5 Web Audio API. This guarantees multi-sensory notification without requiring external audio hardware on the monitoring terminal.

11. Comprehensive RESTful API Reference

The server exposes an organized RESTful interface allowing seamless orchestration between the edge microcontroller, the inference engine, and the web telemetry client:

Endpoint	Method	Payload / Params	Return	Description
/	GET	None	HTML	Serves Web Telemetry Dashboard
/video_feed	GET	None	Stream	MJPEG stream with detection overlays
/api/status	GET	None	JSON	System telemetry & connection state
/api/detections	GET	None	JSON	Active bounding boxes & foot points
/api/snapshot	GET	None	JPEG	Clean, unannotated high-res capture
/api/fence/get	GET	None	JSON	Current polygon vertices list
/api/fence/set	POST	{"polygon": [[x,y],...]}	JSON	Updates virtual fence boundaries
/api/fence/clear	POST	None	JSON	Purges active virtual fence
/api/yolo/toggle	POST	None	JSON	Toggles AI inference loop
/api/yolo/confidence/<val>	POST	Float 0.0 - 1.0	JSON	Updates detection threshold
/api/esp32/buzzer/on	GET	None	JSON	Commands buzzer ON (Port 81)
/api/esp32/buzzer/off	GET	None	JSON	Commands buzzer OFF (Port 81)
/api/alert/toggle	POST	None	JSON	Toggles automatic alarm engine
/api/alert/test	POST	None	JSON	Executes 3-second test alert cycle
/api/logs	GET	None	JSON	Retrieves historical intrusion events
/api/logs/clear	POST	None	JSON	Clears intrusion log database
/api/logs/image/<file>	GET	Filename param	JPEG	Serves evidentiary burst photograph

12. Experimental Benchmarks & Quantitative Evaluation

Testing was conducted across standard edge-to-server deployment environments (AMD Ryzen 7 5800H CPU, NVIDIA RTX 3060 Laptop GPU, 2.4 GHz 802.11n Wi-Fi):

12.1 Latency Analysis

Processing Stage	CPU Latency	CUDA GPU Latency
MJPEG Stream Ingestion & Decode	12.1 ms	8.4 ms
YOLOv8n Tensor Forward Pass	24.8 ms	5.9 ms
Ray-Casting Point-in-Polygon (10 vertices)	< 0.05 ms	< 0.05 ms
Pinhole Distance Estimation	< 0.01 ms	< 0.01 ms
Buzzer Actuation Latency (Port 81 Socket)	8.2 ms	8.2 ms
Total End-to-End Pipeline Latency	37.0 ms	14.4 ms
Effective Stream Framerate	27.0 FPS	30.0 FPS

12.2 Model Accuracy Metrics

Model Config	Target Entity	Precision	Recall	mAP@0.50	mAP@0.95
Baseline YOLOv8n	Generic Person	0.854	0.812	0.842	0.521
Fine-Tuned Sentinel-Eye	Resident (owner)	0.951	0.932	0.946	0.694

13. Security Hardening & Privacy Preservation

- 1. Zero-Secret Architecture:** All private SMTP passwords and network credentials are segregated into .env (Python) and secrets.h (C++), shielded by comprehensive .gitignore rules.
- 2. File Size Compliance:** Heavy neural network weights (> 100 MB) are excluded from Git version control to comply with repository hosting policies.
- 3. Forensic Privacy Guard:** Test images captured during surveillance experimentation are kept local to prevent unintentional public biometric disclosure.

14. Conclusion & Future Trajectories

14.1 Architectural Synthesis & Impact

SENTINEL EYE demonstrates that high-performance, active cyber-physical security systems can be engineered by synthesizing inexpensive IoT edge sensors with modern convolutional neural networks and computational geometry. By decoupling real-time video capture from neural inference, the framework achieves sub-15ms inference latencies on commodity GPU accelerators while preserving sub-\$10 edge hardware affordability.

14.2 Future Research Frontiers

- Stereo Vision / Dual-Camera Fusion:** Enhancing depth estimation accuracy without monocular anthropometric height constraints.
- Edge TinyML Deployment:** Quantizing lightweight int8 models directly onto next-generation ESP32-S3 or Kendryte K210 vector microcontrollers.
- WebRTC Sub-Second Streaming:** Transitioning from MJPEG over HTTP to ultra-low-latency WebRTC data channels.

14.3 System Requirements Matrix

Subsystem	Minimum Requirement	Recommended Production
Host CPU / Compute	Quad-Core x86_64 (2.0 GHz)	8-Core CPU + NVIDIA RTX GPU (CUDA 12+)
Host Memory / Storage	8 GB RAM, 10 GB SSD Storage	16 GB RAM, 50 GB NVMe Storage
Edge Microcontroller	AI-Thinker ESP32-CAM (4MB PSRAM)	ESP32-CAM + External 2.4GHz Antenna
Network Bandwidth	802.11 b/g/n (5 Mbps uplink)	Dedicated 2.4 GHz IoT Wi-Fi VLAN (15+ Mbps)

ACADEMIC INSTITUTION & OPEN SOURCE REPOSITORY

Institution: Posts and Telecommunications Institute of Technology (PTIT) — Faculty of Information Technology
Repository: [git@github.com:Cheesenoice/loT-EdgeAI-Intrusion-Detection.git](https://github.com/Cheesenoice/loT-EdgeAI-Intrusion-Detection.git)
Author & Lead Engineer: Huynh Huu Tri | Academic Year: 2026 | Stable Production Release

Report published under the MIT Open Source License. Copyright (C) 2026 Sentinel Eye Project.